


**Laboratorium**
**Robotika dan Sistem Cerdas**
**Departemen Teknik Komputer**
***Institut Teknologi Sepuluh Nopember***

# Laporan Sementara
# Praktikum Sistem Mikroprosesor dan Sistem Tertanam

## Pengantar Mikrokontroler

Yoga Andreas Hutajulu - 5024241021

2026

# 1 Pendahuluan

## 1.1 Latar Belakang

Perkembangan teknologi sistem tertanam (*embedded system*) saat ini telah merambah ke berbagai bidang kehidupan, mulai dari perangkat elektronik konsumen, otomasi industri, Internet of Things (IoT), hingga sistem robotika. Di jantung setiap sistem tertanam terdapat mikrokontroler — sebuah komputer berukuran kecil dalam satu chip yang mengintegrasikan prosesor, memori, dan periferal input/output. Pemahaman mendalam terhadap arsitektur dan cara memprogram mikrokontroler menjadi kompetensi fundamental bagi mahasiswa Teknik Komputer.

Dalam praktikum ini, dua platform mikrokontroler yang umum digunakan akan dipelajari, yaitu STM32 Blackpill (berbasis ARM Cortex-M) dan ESP32 (berbasis Xtensa LX6 dengan konektivitas Wi-Fi dan Bluetooth). Kedua platform ini mewakili dua pendekatan pemrograman yang berbeda: framework Arduino yang menekankan kemudahan dan abstraksi tinggi, serta framework ESP-IDF (*Espressif IoT Development Framework*) yang memberikan kendali penuh terhadap perangkat keras melalui API *low-level*.

Selain aspek pemrograman, praktikan juga perlu memahami konsep dasar elektronika digital yang erat kaitannya dengan mikrokontroler, seperti konfigurasi resistor *pull-up* dan *pull-down*, fenomena *bouncing* pada tombol mekanik, serta prinsip kerja ADC (*Analog-to-Digital Converter*) dalam membaca sinyal analog. Penguasaan konsep-konsep ini akan menjadi bekal penting dalam merancang dan mengembangkan sistem tertanam yang andal pada modul-modul praktikum selanjutnya.

## 1.2 Dasar Teori

### 1.2.1 PlatformIO dan VSCode sebagai Lingkungan Pengembangan

PlatformIO adalah ekosistem *open-source* untuk pengembangan sistem tertanam yang terintegrasi dengan Visual Studio Code (VSCode) sebagai ekstensi. PlatformIO mendukung lebih dari 1.500 board mikrokontroler dari berbagai vendor dan menyediakan sistem manajemen *library*, konfigurasi *build* otomatis, serta fitur *debugging*. Dibandingkan dengan Arduino IDE konvensional, PlatformIO menawarkan lingkungan pengembangan yang lebih profesional dengan fitur *autocomplete*, *syntax highlighting* yang lebih baik, dan dukungan multi-proyek. Konfigurasi proyek dilakukan melalui file `platformio.ini` yang mendefinisikan target board, framework, serta dependensi library.

### 1.2.2 STM32 Blackpill dan Framework Arduino

STM32 Blackpill adalah board pengembangan berbasis mikrokontroler STM32F411CEU6 (ARM Cortex-M4) dengan clock hingga 100 MHz, 512 KB Flash, dan 128 KB SRAM. Board ini tidak memiliki chip USB-to-Serial *on-board*, sehingga memerlukan programmer eksternal ST-Link untuk proses *flashing* dan *debugging*. Dengan framework Arduino, pemrograman STM32 menjadi lebih sederhana karena menggunakan fungsi-fungsi abstraksi tinggi seperti `pinMode()`, `digitalWrite()`, dan `analogRead()`, meskipun dengan trade-off berupa berkurangnya kendali langsung terhadap register perangkat keras.

### 1.2.3 ESP32 dan Framework ESP-IDF

ESP32 adalah mikrokontroler dari Espressif Systems berbasis prosesor dual-core Xtensa LX6 dengan clock hingga 240 MHz, dilengkapi Wi-Fi 802.11 b/g/n dan Bluetooth 4.2/BLE secara terintegrasi. Ber-

beda dengan STM32 Blackpill, ESP32 memiliki chip USB-to-Serial (biasanya CP2102 atau CH340) sehingga dapat diprogram langsung melalui kabel USB tanpa programmer eksternal. ESP-IDF merupakan framework resmi dari Espressif yang menggunakan bahasa C dan menyediakan akses *low-level* ke seluruh fitur perangkat keras ESP32, termasuk FreeRTOS sebagai sistem operasi *real-time*.

### 1.2.4 Konfigurasi Pull-up dan Pull-down Resistor pada Tombol

Resistor *pull-up* dan *pull-down* digunakan untuk memastikan pin input GPIO memiliki kondisi logika yang pasti (HIGH atau LOW) saat tidak ada sinyal aktif. Pada konfigurasi **pull-up**, resistor menghubungkan pin input ke VCC sehingga kondisi *default* (tombol tidak ditekan) adalah HIGH, dan saat tombol ditekan pin akan terhubung ke GND (LOW). Sebaliknya, pada konfigurasi **pull-down**, resistor menghubungkan pin input ke GND sehingga kondisi *default* adalah LOW, dan saat tombol ditekan pin akan terhubung ke VCC (HIGH). Tanpa resistor ini, pin input akan berada dalam kondisi *floating* yang menghasilkan pembacaan tidak stabil.

### 1.2.5 Debouncing

*Bouncing* adalah fenomena osilasi cepat antara kondisi HIGH dan LOW yang terjadi saat kontak mekanik pada tombol bergetar selama proses penekanan atau pelepasan. Fenomena ini dapat menyebabkan mikrokontroler mendeteksi satu penekanan tombol sebagai beberapa penekanan berturut-turut. Teknik *debouncing* dapat dilakukan secara perangkat lunak (*software*) dengan menambahkan *delay* atau menggunakan algoritma pengecekan waktu (*timestamp-based*), maupun secara perangkat keras (*hardware*) dengan menambahkan kapasitor paralel pada tombol untuk meredam osilasi.

### 1.2.6 ADC sebagai Mode Akses GPIO Ketiga

ADC (*Analog-to-Digital Converter*) memungkinkan mikrokontroler membaca sinyal analog dan mengubahnya menjadi nilai digital. Proses konversi dilakukan dengan cara *sampling* tegangan analog pada interval tertentu dan mengkuantisasinya ke dalam level diskret berdasarkan resolusi bit ADC. Resolusi ADC menentukan presisi pembacaan: ADC 10-bit menghasilkan $2^{10} = 1024$ level, ADC 12-bit menghasilkan $2^{12} = 4096$ level. Semakin tinggi resolusi bit, semakin detail dan akurat representasi digital dari sinyal analog yang terbaca. ESP32 memiliki ADC 12-bit, sedangkan STM32F411 juga memiliki ADC 12-bit.

## 2 Tugas Pendahuluan

Berikut adalah jawaban dari tugas pendahuluan Modul 1: Pengantar Mikrokontroler.

1. **Perbedaan mendasar antara framework Arduino dan ESP-IDF dalam memprogram mikrokontroler:**

   **Framework Arduino** merupakan framework yang dirancang untuk kemudahan penggunaan dengan tingkat abstraksi tinggi. Arduino menyediakan fungsi-fungsi sederhana seperti `pinMode()`, `digitalWrite()`, `analogRead()`, dan `delay()` yang menyembunyikan kompleksitas register perangkat keras dari pengguna. Struktur programnya menggunakan dua fungsi utama: `setup()` dan `loop()`.

**Kelebihan Arduino:**

- Mudah dipelajari, cocok untuk pemula
- Banyak library pihak ketiga yang tersedia
- Portabel lintas berbagai platform mikrokontroler (AVR, ARM, ESP32, STM32)
- Komunitas besar dan dokumentasi melimpah

**Kekurangan Arduino:**

- Abstraksi tinggi mengurangi kendali langsung terhadap perangkat keras
- Kurang optimal dari segi performa dan penggunaan memori
- Tidak mendukung fitur lanjutan seperti RTOS secara native
- Tidak ideal untuk aplikasi *real-time* yang membutuhkan presisi tinggi

**Framework ESP-IDF** merupakan framework resmi dari Espressif untuk ESP32 yang menggunakan bahasa C/C++ dan menyediakan akses langsung ke API *low-level* serta fitur FreeRTOS bawaan.

**Kelebihan ESP-IDF:**

- Kendali penuh terhadap perangkat keras dan register
- Mendukung FreeRTOS untuk *multitasking* dan aplikasi *real-time*
- Performa dan efisiensi memori lebih optimal
- Akses ke seluruh fitur ESP32 (Wi-Fi, Bluetooth, deep sleep, dll.)

**Kekurangan ESP-IDF:**

- Kurva pembelajaran lebih tinggi
- Kode lebih panjang dan kompleks untuk tugas sederhana
- Hanya mendukung keluarga chip ESP (tidak portabel)
- Dokumentasi lebih teknis dan membutuhkan pemahaman arsitektur perangkat keras

2. **Mengapa STM32 Blackpill memerlukan programmer eksternal (ST-Link) sedangkan ESP32 dapat diprogram langsung melalui USB:**

   Perbedaan ini berkaitan langsung dengan keberadaan chip **USB-to-Serial** pada board. Board ESP32 (seperti DevKit) memiliki chip USB-to-Serial bawaan (*on-board*), umumnya CP2102 atau CH340, yang berfungsi sebagai jembatan komunikasi antara port USB komputer dan pin UART (TX/RX) pada mikrokontroler ESP32. Chip ini mengonversi sinyal USB menjadi sinyal serial UART, sehingga komputer dapat langsung mengirimkan firmware ke ESP32 melalui protokol *bootloader* serial bawaan.

   Sebaliknya, STM32 Blackpill **tidak memiliki chip USB-to-Serial** di boardnya. Meskipun memiliki port USB Type-C (yang terhubung ke periferal USB OTG pada STM32F411), port ini tidak dikonfigurasi secara default untuk keperluan *flashing* firmware. Oleh karena itu, diperlukan programmer eksternal **ST-Link** yang berkomunikasi melalui antarmuka SWD (*Serial Wire Debug*) untuk mem-*flash* dan men-*debug* firmware pada STM32. ST-Link menggunakan protokol debug khusus ARM yang berbeda dari komunikasi serial biasa.

3. **Perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor:**

   **Konfigurasi Pull-Up Resistor:**

   - Resistor dihubungkan antara pin input GPIO dan VCC (tegangan suplai)
   - **Saat tombol tidak ditekan:** Pin input ditarik ke VCC melalui resistor, sehingga terbaca **HIGH** (logika 1)
   - **Saat tombol ditekan:** Pin input terhubung langsung ke GND melalui tombol (jalur impedansi rendah), sehingga terbaca **LOW** (logika 0)
   - Logika bersifat *active-low* — aksi terjadi saat sinyal bernilai LOW

   **Konfigurasi Pull-Down Resistor:**

   - Resistor dihubungkan antara pin input GPIO dan GND
   - **Saat tombol tidak ditekan:** Pin input ditarik ke GND melalui resistor, sehingga terbaca **LOW** (logika 0)
   - **Saat tombol ditekan:** Pin input terhubung langsung ke VCC melalui tombol (jalur impedansi rendah), sehingga terbaca **HIGH** (logika 1)
   - Logika bersifat *active-high* — aksi terjadi saat sinyal bernilai HIGH

   Kedua konfigurasi ini bertujuan untuk menghindari kondisi *floating* pada pin input, yaitu kondisi di mana pin tidak terhubung ke level tegangan manapun sehingga pembacaan menjadi tidak menentu dan rentan terhadap *noise* elektromagnetik.

4. **Fenomena bouncing pada tombol mekanik dan cara mengatasinya:**

   *Bouncing* terjadi karena kontak mekanik pada tombol (*push button*) tidak langsung membuat hubungan sempurna saat ditekan atau dilepas. Secara fisik, pelat kontak logam di dalam tombol mengalami getaran (*vibration*) dan pantulan berulang-ulang selama beberapa milidetik (umumnya 5–50 ms) sebelum akhirnya stabil. Selama periode ini, sinyal yang terbaca oleh mikrokontroler berosilasi antara HIGH dan LOW secara cepat, sehingga satu kali penekanan tombol dapat terdeteksi sebagai beberapa kali penekanan.

   **Minimal dua pendekatan untuk mengatasi *bouncing*:**

   a. **Software Debouncing (Debouncing Perangkat Lunak):** Metode ini menggunakan logika dalam program untuk menyaring osilasi. Pendekatan yang umum digunakan adalah:
      - Menambahkan *delay* setelah deteksi perubahan sinyal (misalnya `delay(50)`) agar osilasi selesai sebelum pembacaan berikutnya
      - Menggunakan pencatatan *timestamp* dengan fungsi `millis()` — program hanya menerima perubahan sinyal jika selisih waktu sejak perubahan terakhir melebihi ambang batas tertentu (*debounce interval*)

   b. **Hardware Debouncing (Debouncing Perangkat Keras):** Metode ini menggunakan komponen elektronik tambahan pada rangkaian:
      - Menambahkan **kapasitor** secara paralel dengan tombol (rangkaian RC *filter*). Kapasitor akan menyerap dan meredam osilasi cepat, menghasilkan transisi sinyal yang lebih halus

- Menggunakan IC Schmitt Trigger yang memiliki *hysteresis* sehingga hanya merespons perubahan tegangan yang melewati ambang batas tertentu

5. **Cara ADC mengubah sinyal analog menjadi nilai digital dan pengaruh resolusi bit:**

   ADC (*Analog-to-Digital Converter*) mengubah sinyal analog (tegangan kontinu) menjadi nilai digital (bilangan diskret) melalui dua proses utama:

   a. **Sampling:** ADC mengambil sampel tegangan analog pada titik-titik waktu tertentu sesuai frekuensi *sampling*. Setiap sampel merepresentasikan nilai sesaat dari sinyal analog.

   b. **Kuantisasi:** Nilai tegangan yang di-*sampling* dipetakan ke level diskret terdekat berdasarkan resolusi bit ADC. Proses ini membagi rentang tegangan referensi (misalnya 0–3.3V) menjadi sejumlah level yang ditentukan oleh jumlah bit.

   **Pengaruh resolusi bit ADC terhadap presisi:**

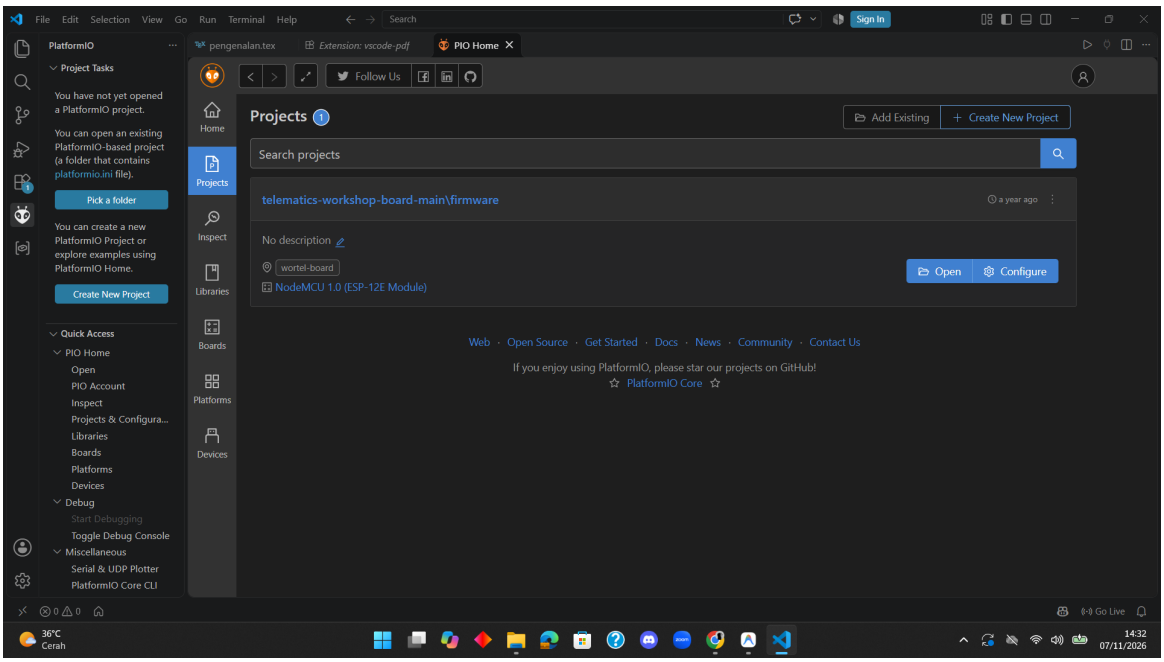
   Resolusi bit menentukan jumlah level diskret yang tersedia, yaitu sebanyak $2^n$ level (dengan $n$ = jumlah bit). Semakin tinggi resolusi, semakin kecil perubahan tegangan terkecil (*least significant bit* / LSB) yang dapat dideteksi:

   - **ADC 10-bit:** $2^{10} = 1024$ level. Dengan tegangan referensi 3.3V, resolusi per step = $3.3V/1024 \approx 3.22$ mV
   - **ADC 12-bit:** $2^{12} = 4096$ level. Dengan tegangan referensi 3.3V, resolusi per step = $3.3V/4096 \approx 0.81$ mV
   - **ADC 16-bit:** $2^{16} = 65536$ level. Dengan tegangan referensi 3.3V, resolusi per step = $3.3V/65536 \approx 0.05$ mV

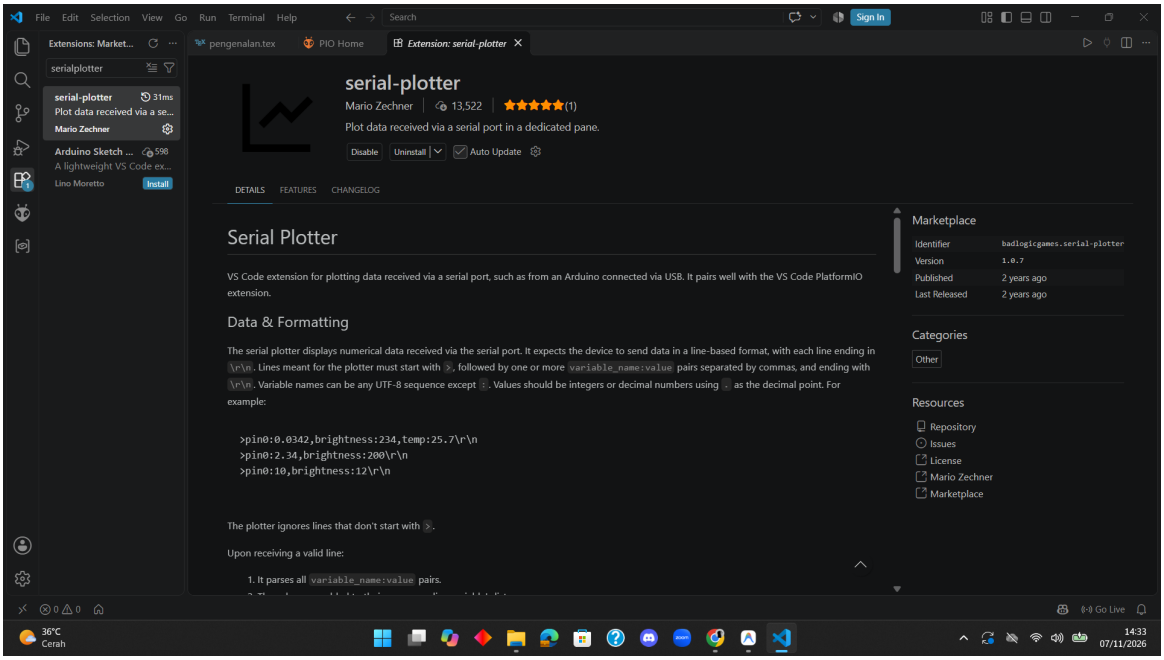
   Semakin tinggi resolusi bit ADC, semakin presisi dan akurat representasi digital dari sinyal analog, sehingga perubahan tegangan yang sangat kecil pun dapat terdeteksi. Namun, ADC dengan resolusi lebih tinggi umumnya memiliki waktu konversi yang lebih lama dan biaya yang lebih mahal.

6. **Set-up tools: VSCode, ekstensi PlatformIO, dan ekstensi Serial Plotter:**

   Berikut adalah bukti *screenshot* bahwa tools telah ter-*set-up* dengan baik pada laptop:




**Gambar 1:** Ekstensi PlatformIO IDE telah terinstal pada VSCode




**Gambar 2:** Ekstensi Serial Plotter telah terinstal pada VSCode

Dari kedua *screenshot* di atas, dapat dilihat bahwa VSCode telah dilengkapi dengan ekstensi PlatformIO IDE (dengan framework ESP-IDF dan dukungan STM32) serta ekstensi Serial Plotter untuk memvisualisasikan data serial dari mikrokontroler.